Intitulé du cours: Technique des applications web

Framework

Filière: 2éme année – bts web

Enseignante: Abid Arij

Année universitaire 2025/2026

Objectifs

- Maitriser l'architecture du framework **django**
- Développer une application web avec le framework **django**

Présentation de Django

- Django est un framework de développement web **open-source** écrit en **Python**.
- Il a été conçu pour simplifier le processus de développement d'applications web robustes, sécurisées et évolutives.
- **Caractéristiques de Django**
  - MVT (Modèle-Vue-Template)
  - ORM (Object-Relational Mapping)
  - Administration automatique
  - Système de routage (URLs)
  - Gestion des formulaires
  - Authentification et autorisation
  - API REST (Django Rest Framework)

django

Framework web Python

Développer rapidement des applications web

Avec le minimum de code

Développé entre 2003 et 2005

À quoi sert Django ?

Django sert à créer des applications web côté **serveur (back-end)** :

- Sites web classiques (blogs, e-commerce, forums…)
- Applications web complexes avec base de données
- APIs REST pour applications mobiles ou front-end modernes (React, Angular, Vue)

Avantages de Django

- **Rapide** : framework conçu pour le développement rapide.
- **Sécurisé** : protège contre SQL injection, XSS, CSRF…
- **Complet** : contient tout ce qu'il faut (authentification, ORM, routing…).
- **Scalable** : peut gérer des applications très grandes.
- **Communauté active** : beaucoup de documentation et d'extensions.
- **ORM intégré** : manipule la base de données sans SQL direct.

Caractéristiques de Django

- Framework **back-end** basé sur Python
- Architecture **MVT**
- Système de **routing / URL dispatcher**
- Gestion intégrée des **bases de données**
- **Moteur de templates** pour générer le HTML
- Gestion de la **sécurité**, **sessions**, **authentification**, **admin interface**
- Extensible via des **applications Django** et des packages externes

- Entreprises utilisant Django Framework

Environment Variables en Django

Les **variables d'environnement** sont des paramètres qu'on configure **à l'extérieur du code**, souvent dans le système ou un fichier .env.

Elles servent à :

- Stocker des infos sensibles (ex: SECRET_KEY, mots de passe, clés API).
- Adapter ton projet selon l'environnement (développement, test, production).

Environnement virtuel 1/4

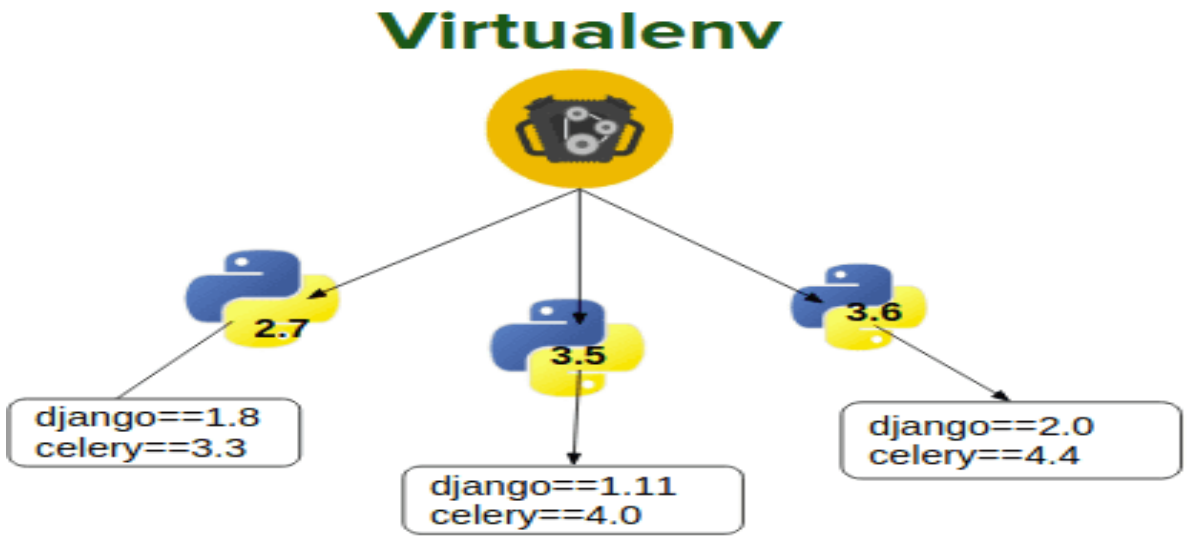
Un **environnement virtuel** (ou *virtual environment*, abrégé **venv**) est **un espace isolé** sur ton ordinateur où tu installes **les dépendances (librairies)** nécessaires à un projet Python.

👉 C'est comme une **boîte séparée** contenant :

ton interpréteur Python,

tes modules installés (comme Django, NumPy, etc.),

sans interférer avec les autres projets.



Environnement virtuel 2/4

Imaginons que tu travailles sur deux projets :

Projet	Besoin de Django	Autres modules
Projet A	Django 4.2	Pillow 9.0
Projet B	Django 5.0	requests 2.31

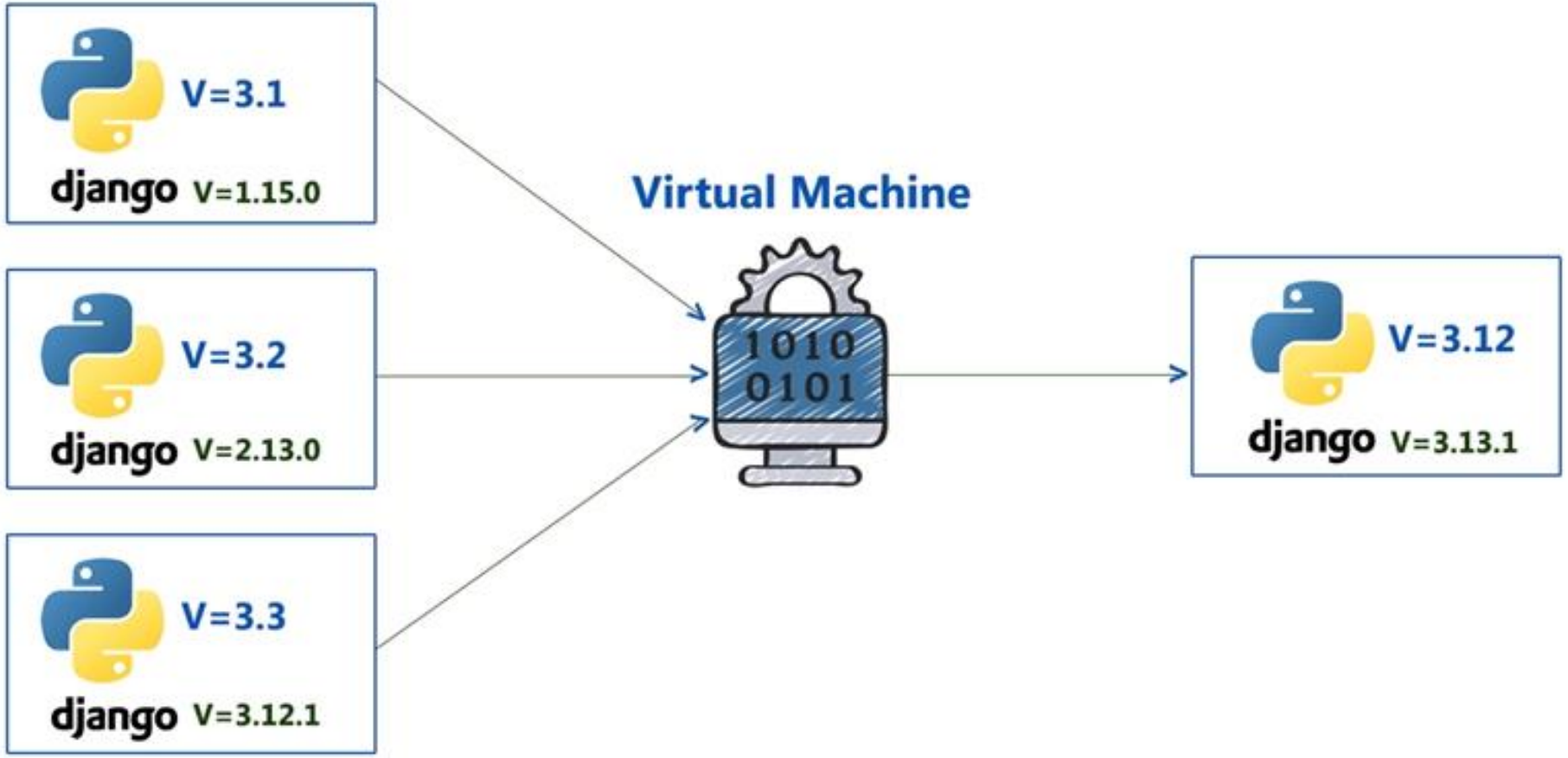
- Sans environnement virtuel :
  - Tous les projets utiliseraient les **mêmes versions** de bibliothèques installées sur ton système.
  - Si tu mets Django à jour pour le projet B, tu risques de **casser le projet A**. ⚠
- Avec un environnement virtuel :
  - Chaque projet a **sa propre version de Django**.
  - Ils sont **indépendants** l'un de l'autre

Environnement virtuel 3/4

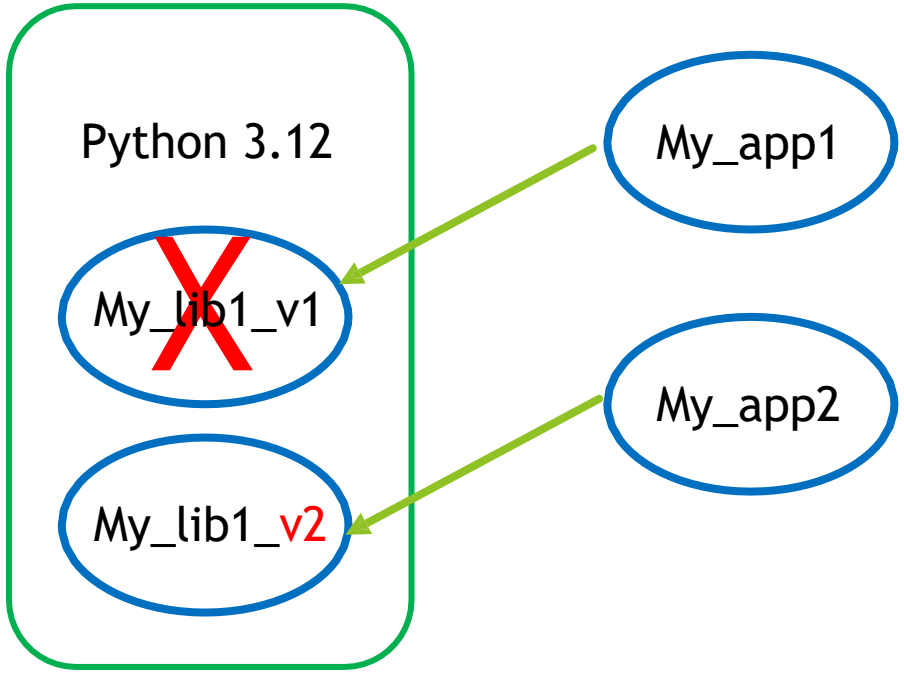
En résumé : les avantages

Avantage	Description
Isolation	Chaque projet a ses propres bibliothèques, sans conflit.
Organisation	Tu gardes un environnement propre, sans polluer ton système.
Portabilité	Tu peux facilement partager ton projet avec un fichier requirements.txt.
Expérimentation sûre	Tu peux tester différentes versions de Django sans risquer de tout casser.

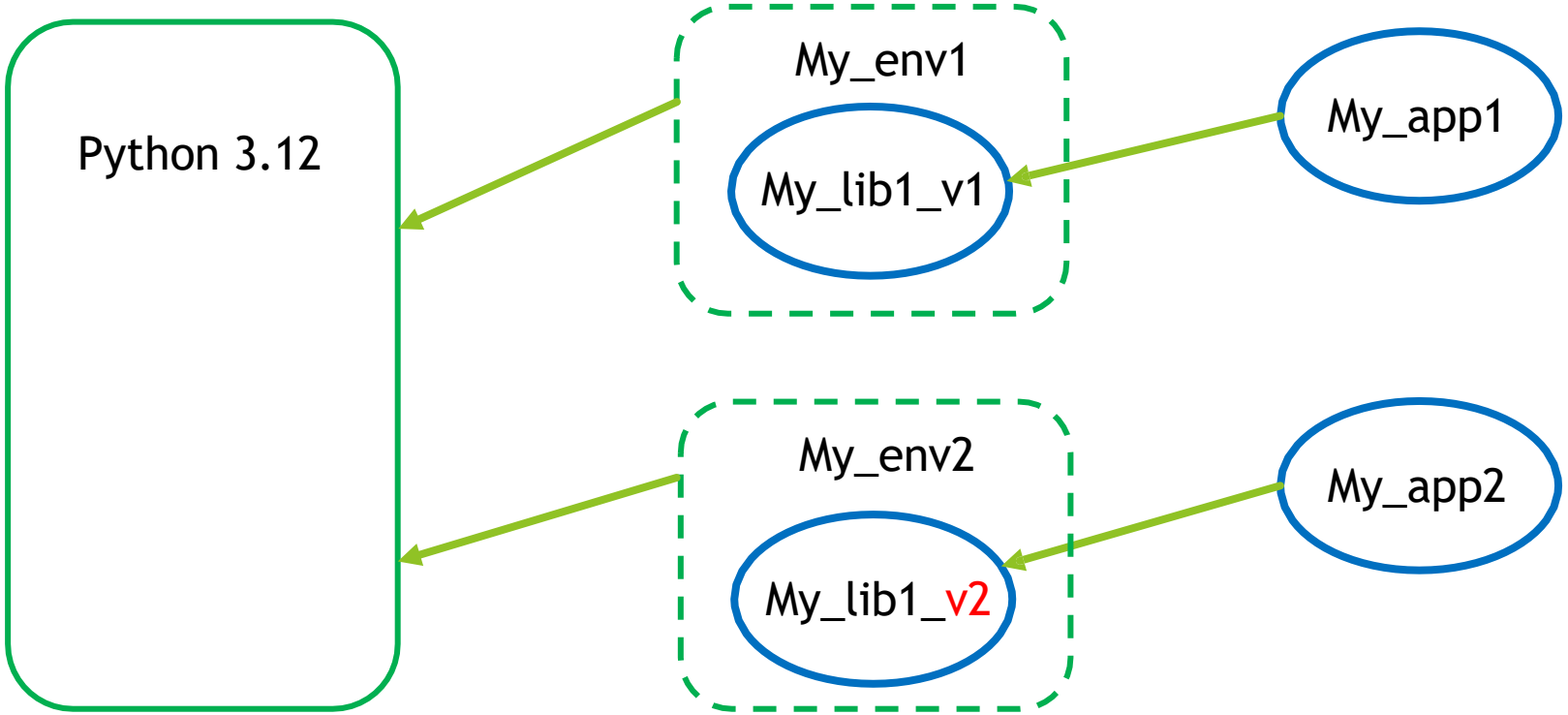
Environnement virtuel 3/4



Environnement virtuel: problématique



Environnement virtuel: solution



Environnement virtuel

- Ne pas installer les packages python globalement
- Toujours travailler dans un environnement virtuel
- Éviter les conflits de dépendances
- Travailler dans un contexte isolé.

pip = *Python Installer Package*.

- C’est le **gestionnaire de paquets** officiel de Python.
- Il permet d’installer des librairies externes (ex: Django, NumPy, Pandas…

Django : framework backend

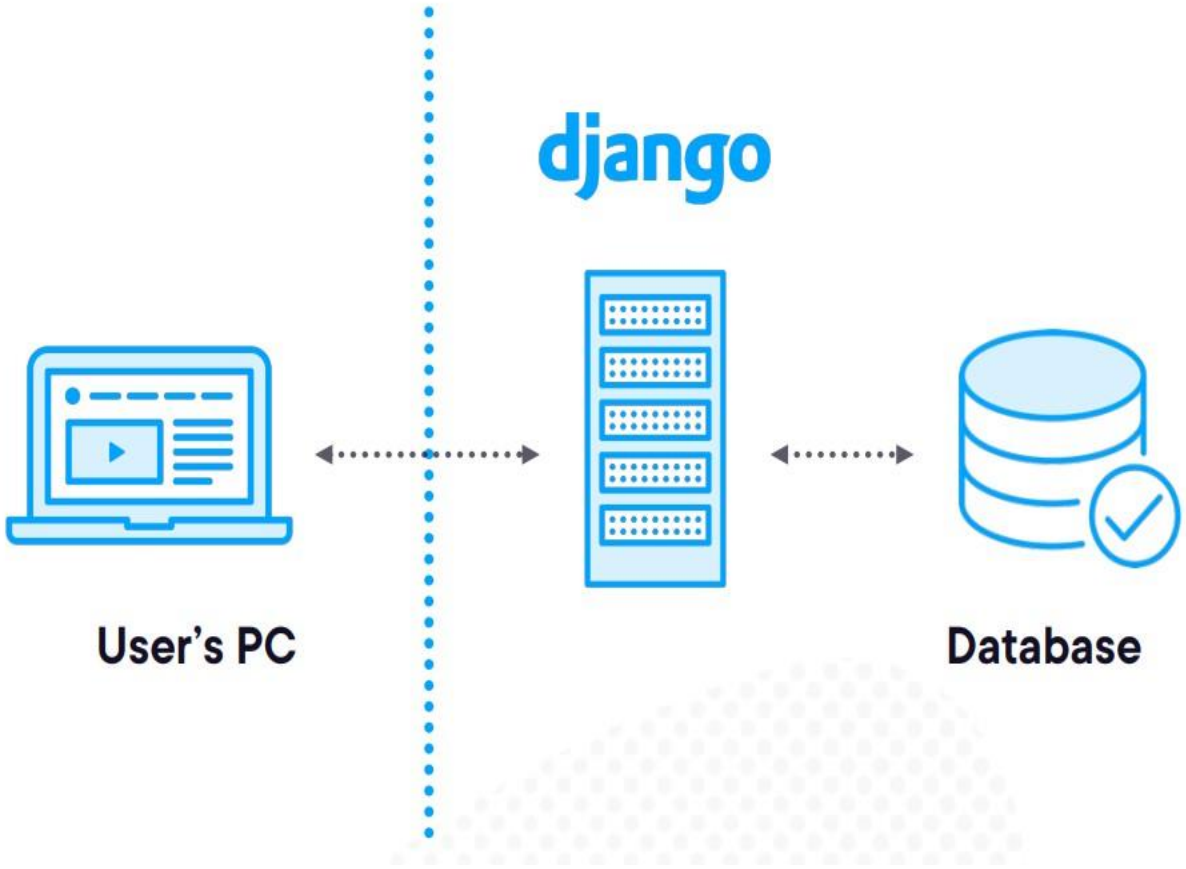
Django runs on backend

Server side

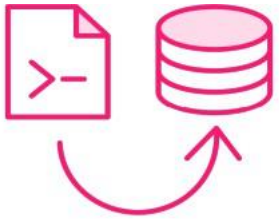
Read/write to DB

Render HTML pages

Combine with React, Angular etc.



Fonctionnalités de Django

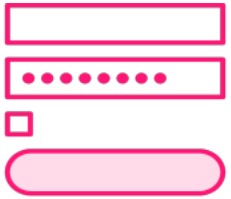


Data model Layer

Admin interface

Template language

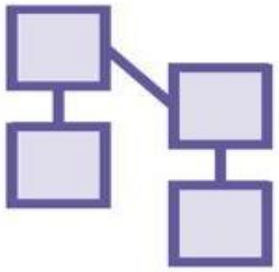
URL mapping



Forms

Packages

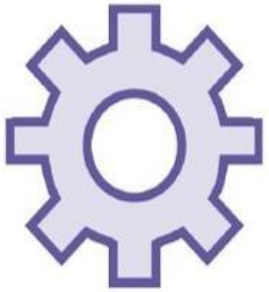
django: Architecture MVT



Model

Représente les données,
Assure le mapping
Objet/Relationnel

MVC: "Controller"



View

Reçoie une requête http, effectue un traitement, retourne une réponse http

MVC: "View"

Template

Définit, génère la présentation HTML

django: Architecture MVT 1/2

- **Modèle (Model)**
- Représente la **structure de la base de données**.
- Décrit les **tables, champs, relations** entre les données.
- Exemples : utilisateurs, articles, produits.
- Django utilise son **ORM** pour interagir avec la base de données.

- **Template (Template)**

Gère le **front-end / affichage**.

Ce sont les **fichiers HTML** avec des balises spéciales pour intégrer les données du back-end.

Exemple : afficher une liste d'articles dans une page web.

django: Architecture MVT 2/2

- **Vue (View)**
- C’est la **logique entre le modèle et le template**.
- Elle reçoit une **requête HTTP**, traite les données via le **Model**, et retourne une **réponse HTML** via le **Template**.

- **URL Dispatcher**
- Gère la **route de chaque page**.
- Associe **l’URL** à la **vue correspondante**.

Django utilise l'architecture **MVT (Model – View – Template)** qui est une variante du **MVC (Model – View – Controller)**.

1. **MVC** :
   a. **Model** → représente les données (ex: tables en base de données).
   b. **View** → gère l'affichage, ce que l'utilisateur voit.
   c. **Controller** → reçoit les requêtes de l'utilisateur, exécute la logique et renvoie une vue.
2. **MVT** (version Django) :
   a. **Model** → pareil que dans MVC, gère les données.
   b. **Template** → gère la partie **affichage** (HTML/CSS).
   c. **View** → en Django, c'est ce qui joue le rôle du **Controller** → reçoit la requête, appelle le **Model**, envoie la réponse au **Template**.
3. 👉 La différence principale :

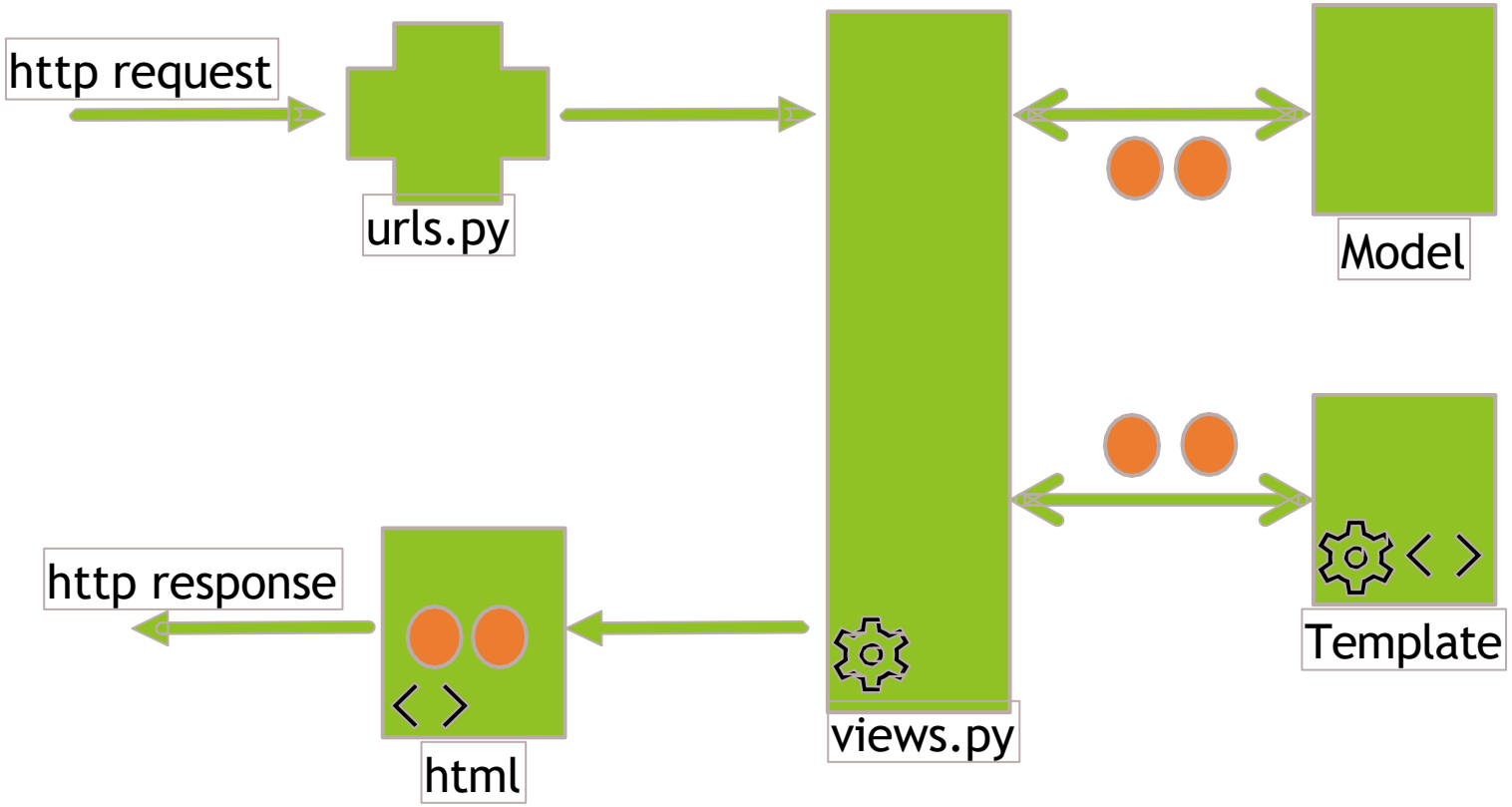
   Dans Django, la **Vue (View)** n'affiche pas directement l'interface, elle sert de **contrôleur**. Le **Template** s'occupe de l'affichage.

...

django: Architecture MVT

1. **L'utilisateur** visite une page (ex : /produits/)
2. **L'URL** appelle une **vue**
3. La **vue** récupère les **données** du **modèle**
4. La **vue** envoie ces données au **template**
5. Le **template** affiche le résultat à l'utilisateur

➡ C'est ce qu'on appelle le **flux MVT**



Système d'exploitation: windows, Linux, Mac OS

Python: Latest version 3.13.7

Éditeur de code: VS code, PyCharm

django: Première application

Étape par Étape

django: Première application

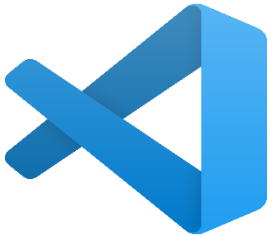
Étape par Étape

- Installation et configuration des outils
- Création et activation d'un environnement virtuel pour le projet
- Installation du framework django (virtual env activé)
- Sauvegarder les dépendances
- Création du projet django nommé **hello_word_project**
- Création de l’application **pages**
- Déclaration de l’application **pages**
- Première page web

Installation et configuration des outils

Étape par Étape

- Installer Python: https://www.python.org/downloads/
- Installer VS Code ou PyCharm



Environnement virtuel

Étape par Étape

```
#créer un dossier pour le projet
$ cd framework_django\projects
$ mkdir helloword
$ cd helloword
```

```
#créer et activer l'environnement virtuel
$ python -m venv my_env
$ my_env\Scripts\activate
(my_env) $ python -m pip install django
```

```
$ source my_env/bin/activate
```

Sauvegarder les dépendances

Étape par Étape

```
# sauvegarder les dépendances

(my_env) $ pip freeze > requirements.txt
```

requirements.txt

```
asgiref==3.8.1
Django==5.1.1
sqlparse==0.5.1
tzdata==2024.1
```

```
# insaller les dépendances

(my_env) $ python -m pip install -r requirements.txt
```

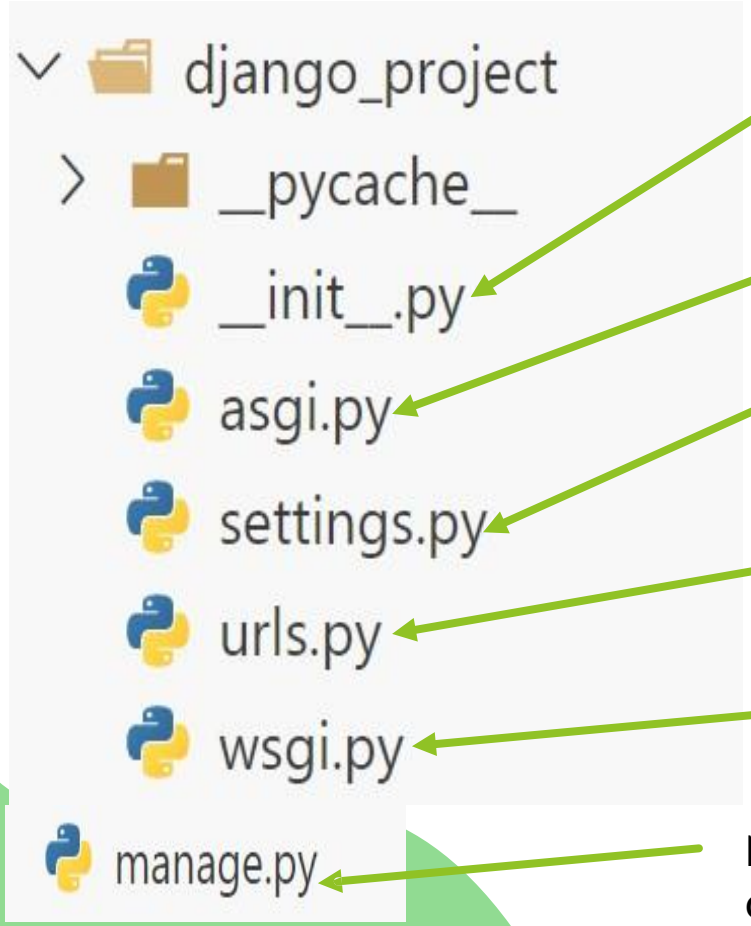
Créer le projet django_project

Étape par Étape

```
# créer le projet django_project
(my_env) $ django-admin startproject django_project  .
# lancer le projet dans VS code
(my_env) $ code .
```

django_project :

Étape par Étape



Marque le répertoire comme package => pouvoir importer les composants

Standard Python pour application et serveur asynchrone

Contrôle l'application django via des configurations

Binding entre requête http et traitement associé

Web Server Gateway Interface

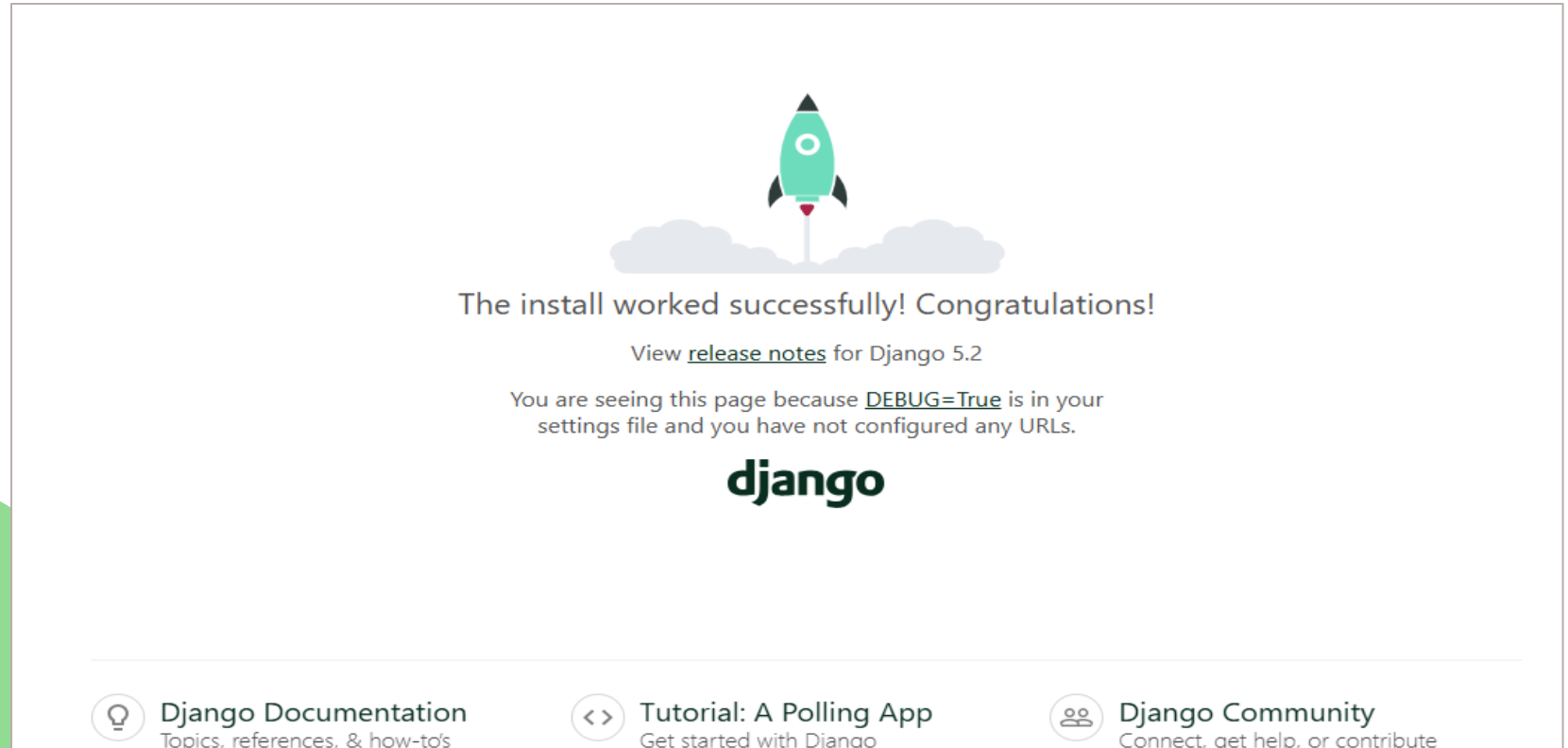
Ne fait pas partie du projet, mais utile pour exécuter des commandes: runserver, créer nouvelle application,…

Lancer le projet

Étape par Étape

```
# démarrer le projet

(my_env) $ python manage.py runserver
```



Différence entre un projet et une application en Django

Le projet Django
Un **projet** Django, c'est **le cadre général** de ton site web.
C'est **le conteneur principal** qui réunit toutes les applications, les fichiers de configuration et les paramètres du site.

- Un projet contient :

les **paramètres globaux** du site (settings.py)
le **fichier des routes principales** (urls.py)
les **fichiers de démarrage** (manage.py, etc.)

L'application Django

Une **application** est une **partie spécifique** du projet qui gère une fonctionnalité précise.
Chaque application fait **une seule chose**.

- **Exemple :**
  Dans ton projet MaBoutiqueEnLigne, tu peux avoir :

une application produits → pour gérer les produits
une application commandes → pour gérer les commandes
une application utilisateurs → pour les profils clients

Chaque application a :

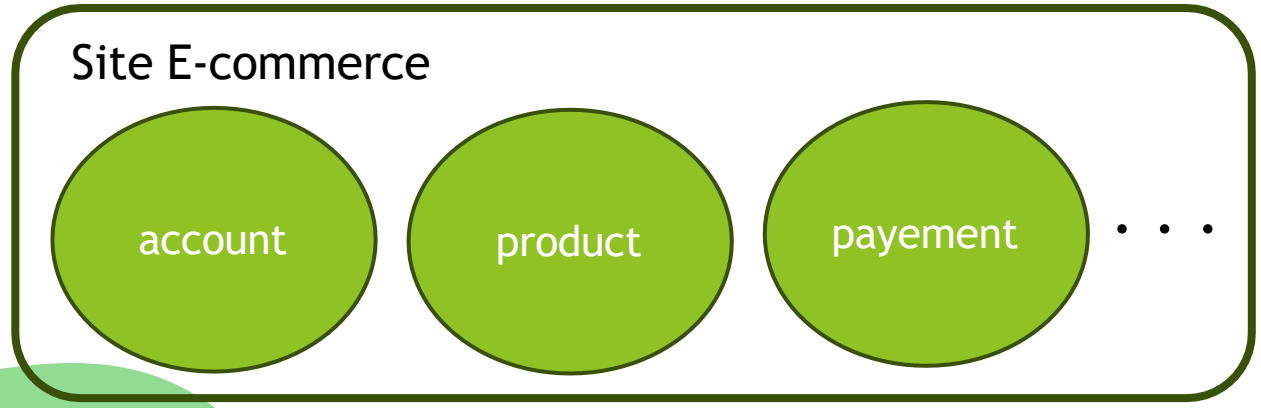
- ses **modèles** (models.py)
- ses **vues** (views.py)
- ses **templates** (HTML)
- et parfois son propre **urls.py**

Ainsi, un **projet = plusieurs applications** qui travaillent ensemble.

Application django: pages

Étape par Étape

- django utilise les concepts **projet** et **application** pour maintenir le code "clean"
- un projet django peut contenir plusieurs applications
- Chaque application contrôle une fonctionnalité isolée du projet.



Application django: pages

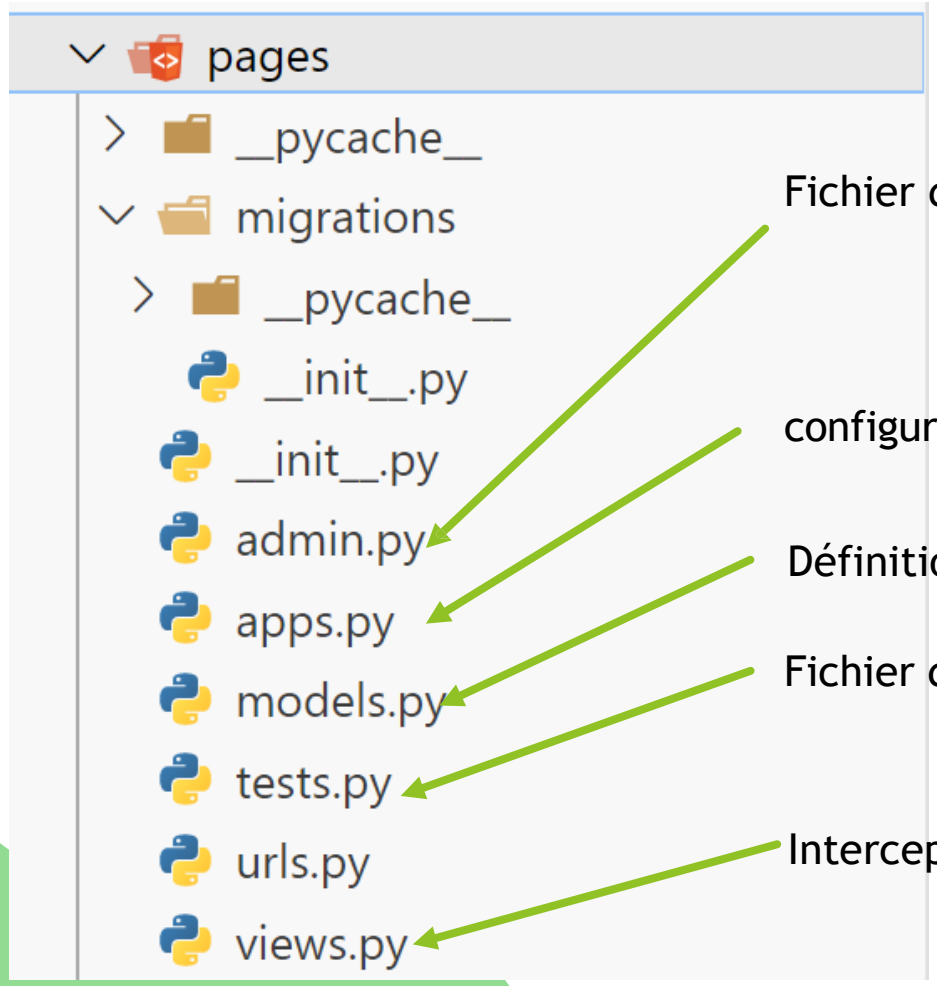
Étape par Étape

```
# créer l'application pages

(my_env) $ python manage.py startapp pages
```

Application django: Anatomie

Étape par Étape



Fichier de configuration pour le module Admin-app

configuration de l'application pages

Définition du modèle

Fichier dans lequel on code les tests

Intercepter les requêtes, traitement, retourner une réponse

Déclaration de l'application pages

Étape par Étape

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    "pages",#new
]
```

Première page web: views.py

Étape par Étape

views.py U

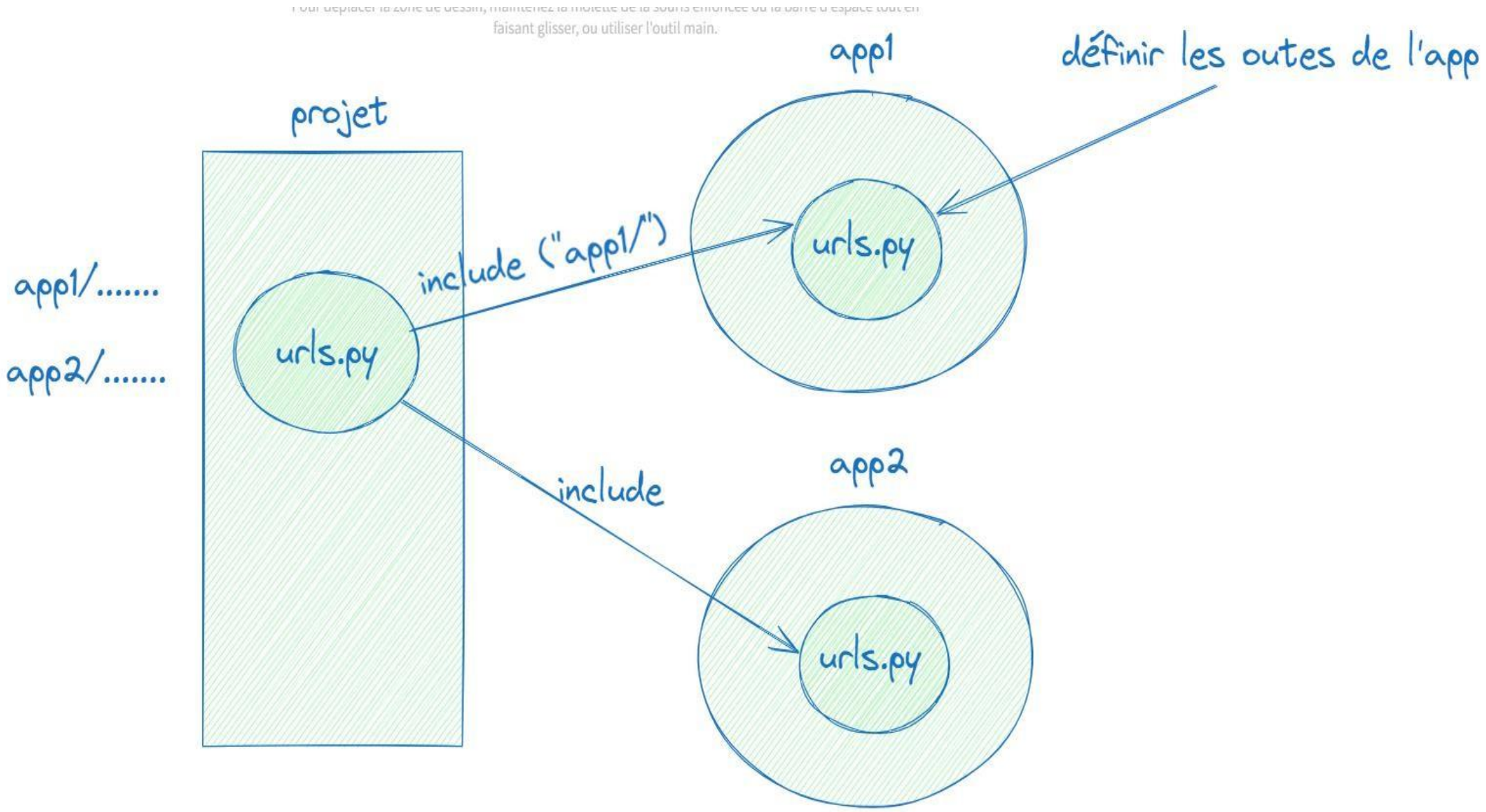
pages > views.py > ...

```python
from django.shortcuts import render
from django.http import HttpResponse

# Create your views here.

def home_page_view(request):
    return HttpResponse("Hello word")

```


Pour déplacer la zone de dessin, maintenez la molette de la souris enfoncée ou la barre d'espace tout en faisant glisser, ou utiliser l'outil main.
app1
définir les outes de l'app
projet
app1/.....
app2/.....
urls.py
include ("app1/")
urls.py
include
app2
urls.py

Première page web: pages.urls.py

Étape par Étape

urls.py U ×

pages > urls.py > ...

```python
from django.urls import path

from .views import home_page_view

#domain.com/pages/.....
urlpatterns=[
    path('home/',home_page_view, name="home"),
]
```

Première page web: urls.py

Étape par Étape

urls.py U ×

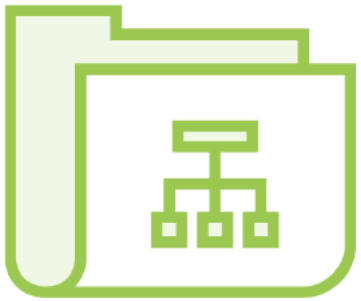
django_project > urls.py > ...

```python
from django.contrib import admin
from django.urls import path,include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('pages/',include("pages.urls")),
]
```

- Django: framework web Python
- Développement web simple et rapide
- Piles incluses: ORM, Admin module, ...
- Architecture MVT (Model, View, Template)
- Un projet django, plusieurs applications
- Virtual environnement

django apps



1. Python package
2. Contient models, views, templates, urls
3. Les projets django contiennent plusieurs apps

   Les Apps peuvent être réutilisées
4. Maintenir les apps petites et simples

Création du projet

Étape par Étape

```
#créer un dossier pour le projet
$ cd framework_django\projects
$ mkdir meetings_planner
$ cd meetings_planner
```

```
#créer et activer l'environnement virtuel
$ python -m venv my_env
$ my_env\Scripts\Activate
(my_env) $ python -m pip install django
```

Sauvegarder les dépendances

Étape par Étape

#sauvegarder les dépendances

(my_env) $ pip freeze > requirements.txt

requirements.txt

requirements.txt

```
1  asgiref==3.7.2
2  Django==4.2.4
3  sqlparse==0.4.4
4  tzdata==2023.3
5
```

Création projet et applications

Étape par Étape

```
# créer le projet django_project
(my_env) $ django-admin startproject meetings_planner .
# lancer le projet dans VS code
(my_env) $ code .
```

```
# créer l'application website
(my_env) $ python manage.py startapp website
```

```
# créer l'application meetings
(my_env) $ python manage.py startapp meetings
```

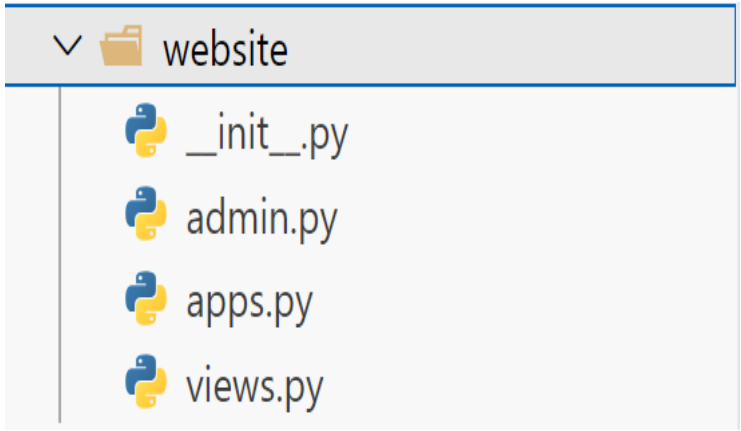
Structure du projet

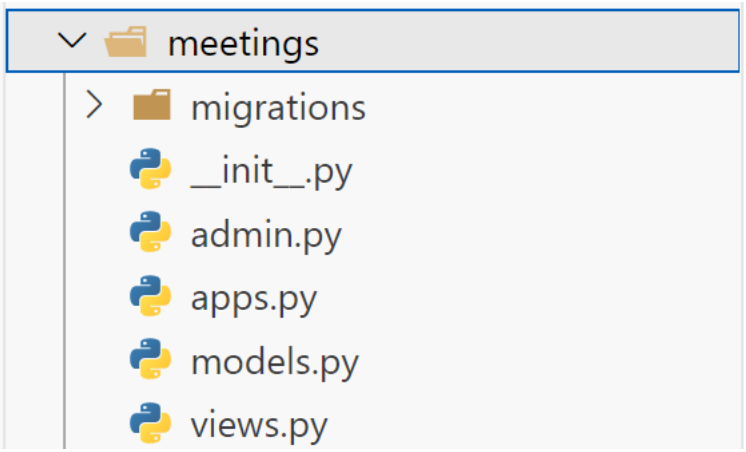
Étape par Étape

- MEETINGS_PLANNER
  - meetings
  - meetings_planner
  - my_env
  - website
  - .gitignore
  - db.sqlite3
  - manage.py
  - requirements.txt

Structure des applications

Étape par Étape





Settings.py: Déclaration des applications

Étape par Étape

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'website',#new
    'meetings',#new
]
```

Merci pour votre attention !